



SDEV2171

Lesson 05: Styling in React Native

A LEADING POLYTECHNIC COMMITTED TO YOUR SUCCESS

Lesson Objectives

By the end of this lesson, students should be able to:

- Apply basic styling techniques to React Native components to improve usability and clarity
- Use `StyleSheet.create` to define and apply component styles
- Apply spacing, alignment, and text styling properties to UI components
- Use layout-related styling to support clear and predictable screen structure
- Debug common styling and layout-related issues in React Native applications

Lesson Roadmap

1. Reconnect lesson-04 structure to visual clarity
2. Introduce key React Native styling patterns
3. Walk through the styling target
4. Guided styling build
5. Experimentation and troubleshooting
6. Checkpoint and lesson-06 handoff

Starting Point From Lesson 04

- use the same Expo project from lesson 04
- open the screen that already uses `View`, `Text`, `Image`, `ScrollView`, and `TextInput`
- confirm the app still runs before changing styles

If your project is not ready

- recover one working lesson-04 baseline first, or
- use the instructor reference app for practice and record one next step for your own project

What Changes Today

- lesson 04 asked “Which component belongs here?”
- lesson 05 asks “How should this structure look and read?”
- the code focus shifts from new components to named style rules

Why Use `StyleSheet.create`

- keeps style rules grouped and reusable
- makes low-level style values easier to scan
- helps tools catch invalid native style properties

Course rule

Use inline styles sparingly. Use `StyleSheet.create` for the main lesson build.

React Native Styling Is Not Web CSS

- style values are JavaScript objects, not CSS strings
- dimensions are unitless numbers
- Flexbox still applies, but defaults differ from the web
- `flexDirection` starts as `column`, not `row`

Plain-language backup

- styles are named settings attached to components
- parent containers still control much of the layout

Quick Comparison

Rough version

- text blocks blend together
- sections feel crowded
- rows do not line up clearly

Clearer version

- title and body text are easier to distinguish
- spacing separates each section
- one row or grouped area has a deliberate alignment rule

Key Property Groups For Today

Purpose	Common properties
Spacing	<code>padding</code> , <code>margin</code> , <code>gap</code>
Text clarity	<code>fontSize</code> , <code>fontWeight</code> , <code>lineHeight</code> , <code>color</code>
Layout	<code>flexDirection</code> , <code>alignItems</code> , <code>justifyContent</code>
Surfaces	<code>backgroundColor</code> , <code>borderWidth</code> , <code>borderColor</code> , <code>borderRadius</code>

Styling Target

- one screen with clearer visual hierarchy
- consistent spacing between sections
- readable text sizes and line spacing
- at least one aligned row or grouped block
- enough visual structure to make later navigation decisions obvious

Still out of scope

- themes
- animation
- deep visual polish
- navigation setup

Recovery Path If Your Project Is Blocked

- if your lesson-04 app runs, stay in your own project
- if `src/app/App.js` is broken, restore the last working version first
- if the project will not launch in time for class progress, open `example/styling-reference/`
- record which project you used and one next step before lesson 06

Recovery app rule

- use it to complete today's styling work
- do not treat every property in the reference app as a requirement

Guided Build Order

1. create a `styles` object with `StyleSheet.create`
2. style the screen container spacing
3. improve text hierarchy
4. style section surfaces
5. adjust one alignment pattern
6. test and keep the cleanest result

Minimal Syntax For Today

```
const styles = StyleSheet.create({  
  container: {  
    padding: 24,  
    gap: 16,  
  },  
  title: {  
    fontSize: 28,  
    fontWeight: '800',  
  },  
});
```

```
<View style={styles.container}>  
  <Text style={styles.title}>Styled title</Text>  
</View>
```

Intermediate Checkpoints

- after container styles: spacing feels more deliberate
- after text styles: headings and body text are easier to distinguish
- after section styles: cards or blocks look grouped clearly
- after alignment changes: rows and section structure feel more predictable

If a checkpoint fails

Stop there and debug that step before adding more styles.

Common Styling Mistakes

- style attached to the wrong component
- parent container missing `flex` or spacing
- text size changes without matching line spacing
- `alignItems` or `justifyContent` changed without understanding the current axis
- too many style edits made before rechecking the app

Parent Container Example

If a row uses `flexDirection: 'row'`

- `justifyContent` moves items left to right
- `alignItems` moves items up and down

If the same container uses `flexDirection: 'column'`

- `justifyContent` moves items top to bottom
- `alignItems` moves items left to right

Debug rule

Check the parent container axis before changing the child component.

Debugging Mini-Lab

Try one controlled test during class:

- temporarily change a row from `flexDirection: 'row'` to `flexDirection: 'column'`
- observe how the layout changes
- restore the original direction and confirm the row returns

Why this matters

It gives you one real layout-debugging loop instead of only reading about styling issues.

Guided Experimentation

- increase or reduce card padding and compare readability
- adjust title and body text sizes
- change one alignment rule on purpose and restore it
- compare one inline style with the equivalent `StyleSheet.create` rule
- keep the cleanest version for the checkpoint

Checkpoint

Students should be ready to show

- a running styled screen
- clearer spacing, readable text, and at least one deliberate alignment choice
- one explanation of why a style change improved usability or clarity

Students should also be able to answer

- Which style group did you apply first?
- What changed most visibly after styling?
- What troubleshooting step helped or would help if the layout broke?

What We Learned Today

- how to use `StyleSheet.create` to organize styling work
- how spacing, text styles, and alignment affect clarity
- how React Native styling differs from web CSS defaults
- how parent containers influence layout
- how to debug styling and layout issues step by step

Next Lesson Bridge

- keep this same project and styled screen
- lesson 06 introduces multi-screen navigation
- next class uses the clearer sections you built today as candidates for separate screens

Ready for lesson 06 means

- the app still runs
- the styled screen is readable and stable
- at least two logical content sections are still clear enough to split into separate screens